

LINE DETECTION

Hough Transform

Hough Transform เป็นวิธีที่ใช้ในการประมวลผลภาพเพื่อตรวจจับรูปร่างใดๆ หากรูปร่างนั้นสามารถแสดงในรูปแบบทางคณิตศาสตร์ได้ สามารถตรวจจับรูปร่างได้แม้จะแตกหักหรือบิดเบี้ยวเล็กน้อยก็ตาม

EDGE DETECTION

Edge Detection

การหาขอบเขตภาพ คือการตรวจสอบว่าเส้นขอบลากผ่านหรือใกล้เคียงกับจุดใด โดยวัด จากการเปลี่ยนแปลงของความเข้มในตำแหน่งที่ใกล้เคียงกับจุดดังกล่าว ซึ่งสามารถที่จะหาขอบภาพได้ด้วยกันหลายวิธี เช่น โซเบล, โรเบิร์ต, พรีวิท, แคนนี่

Sobel Edge Detector

คือการใช้ คอนโวลูชันในการหาขอบของภาพ โดยจะใช้ คอนโวลูชัน G_x ในการหาขอบภาพแนวนอน และ คอนโวลูชัน G_y ในแนวตั้งจากนั้นมารวมกัน

-1	0	+1
-2	0	+2
-1	0	+1

G_x

+1	+2	+1
0	0	0
-1	-2	-1

G_y

Sobel Edge Detector



```
1  sobel = cv2.Sobel(1,2,3,4)
2  # 1 คือ รูปภาพ
3  # 2 คือ ชนิดตัวแปรใน Array(กำหนดค่าเป็น -1 เพื่อให้อ้างอิงกับ Array ของภาพต้นฉบับ)
4  # 3 คือ ตัวแปรในแนวแกน X
5  # 4 คือ ตัวแปรในแนวแกน Y
```

Sobel Edge Detector

```
1  #Sobel Method
2  import cv2
3  import matplotlib.pyplot as plt
4
5
6  img = cv2.imread("image/currency.jpg",0)
7
8  # sobel = cv2.Sobel(1,2,3,4)
9  # 1 คือ รูปภาพ
10 # 2 คือ ขนิตตัวแปรใน Array(กำหนดค่าเป็น -1 เพื่อให้อ้างอิงกับ Array ของภาพต้นฉบับ)
11 # 3 คือ ตัวเกรงในแนวแกน X
12 # 4 คือ ตัวกรองในแนวแกน Y
13 sobelX = cv2.Sobel(img,-1,1,0)
14 sobelY = cv2.Sobel(img,-1,0,1)
15 sobelXY = cv2.bitwise_or(sobelX,sobelY)
16
17 images = [img,sobelX,sobelY,sobelXY]
18 titles = ["Original","SobelX","SobelY","SobelXY"]
19
20 for i in range(len(images)):
21     plt.subplot(2,2,i+1)
22     plt.imshow(images[i],cmap="gray")
23     plt.title(titles[i])
24     plt.xticks([],plt.yticks([]))
25
26 plt.show()
```

Laplacian Method

Laplace Filter เป็นวิธีการทำให้ขอบของภาพมีความคมชัดมากขึ้นโดยการ localizing การเปลี่ยนแปลงอย่างกระทันหันของค่า intensity โดยใช้ second derivatives ของ image function

$$(\nabla^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$$

$$\text{เมื่อ } \frac{\partial^2 f}{\partial^2 x}(x, y) \equiv H_x^L = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix}$$

$$\text{และ } \frac{\partial^2 f}{\partial^2 y}(x, y) \equiv H_y^L = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

$$\text{หรือ } H^L = H_x^L + H_y^L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Laplacian Method



```
1 lap = cv2.Laplacian(1,2)
2 # 1 คือ รูปภาพ
3 # 2 คือ ชนิดตัวแปรใน Array(กำหนดค่าเป็น -1 เพื่อให้อ้างอิงกับ Array ของภาพต้นฉบับ)
```

Laplacian Method

```
1  #Laplacian Method
2  import cv2
3
4  img = cv2.imread("image/currency.jpg",0)
5  # lap = cv2.Laplacian(1,2)
6  # 1 คือ รูปภาพ
7  # 2 คือ ชนิดตัวแปรใน Array(กำหนดค่าเป็น -1 เพื่อให้อ้างอิงกับ Array ของภาพต้นฉบับ)
8
9  lap = cv2.Laplacian(img,-1)
10
11 cv2.imshow("Original",img)
12 cv2.imshow("Laplacian",lap)
13
14 cv2.waitKey(0)
15 cv2.destroyAllWindows()
```

Canny

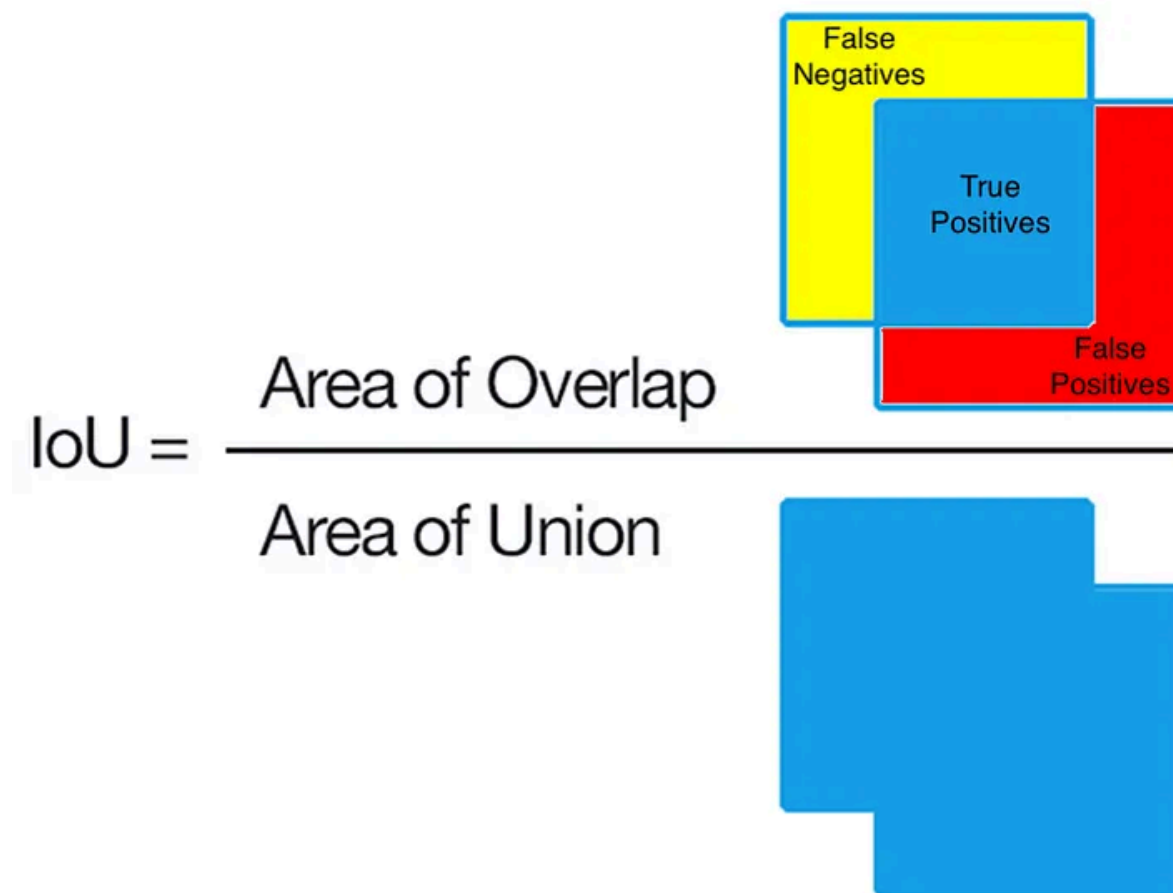
การทำ Canny filter เป็นการทำ Edge detection แบบหนึ่ง โดยจะใช้ multi-stage algorithm ในการตรวจจับขอบภาพ โดยวิธีการนี้จะได้ผลลัพธ์ที่ค่อนข้างมีประสิทธิภาพสูง

การทำ Canny Filter จะมี 4 ขั้นตอนการทำงาน คือ

1. Gaussian blur : Noise Reduction สร้าง filter ในภาพ เพื่อลบรอยและ Noise ในภาพ
2. Gradient operator : หาค่า Intensity Gradients ของรูปภาพ
3. Non-maximum suppression : สร้าง Edge ที่แท้จริง
4. Hysteresis thresholding : หาจุดเริ่มต้น และจุดสุดท้ายของขอบภาพ

Non-maximum suppression

Non-maximum suppression เป็นการนำเอาผลลัพธ์ที่เป็น True Positive ทั้งหมด แล้วลบ Duplicate ออกจากผลลัพธ์ให้เหลือเป็นผลลัพธ์เดียว

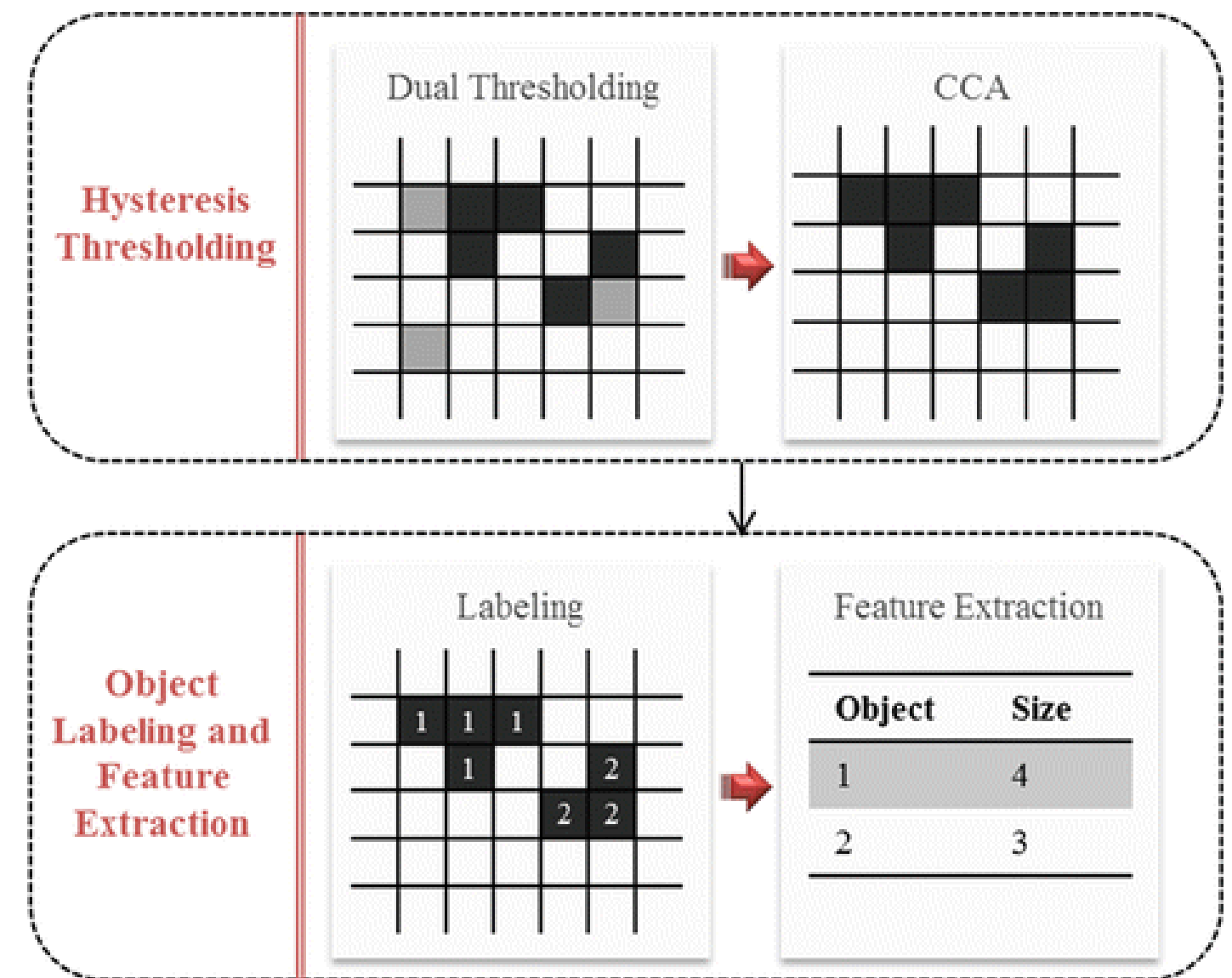


Non-maximum suppression

เพื่อสร้าง Edge ที่แท้จริง เมื่อค่า Pixel นั้นๆ มีความเหมาะสมกับการเป็นขอบมากกว่า Pixel ข้างเคียง หรือสามารถใช้ Modified Sobel ในการหาค่า Max ได้เช่นกัน โดยเลือกจุดภาพที่มีขนาดสูงสุดและมีทิศทางเดียวกับเกรเดียน

Hysteresis thresholding

เพื่อหาจุดเริ่มต้น และจุดสุดท้ายของขอบภาพ โดยการลบเส้นที่ไม่เชื่อมกันออก โดยการพิจารณา connected component หรือ neighborhood บริเวณรอบ หากไม่มี neighborhood ที่เป็น edge เลย (มีค่าไม่เป็น 1) pixel นั้นจะถูก suppress ไว้ วิธีการทำจะตั้ง threshold สองค่าคือ Low และ High หาก pixel นั้นต่ำกว่า Low ค่าจะเป็น 0 ถ้ามากกว่า High จะเป็น 1 หรืออยู่ระหว่างนั้นจะพิจารณาให้เป็น 1 เมื่อมี neighbor เป็น edge



Canny



```
1  canny = cv2.Canny(1,2,3)
2  # 1 คือ รูปภาพ
3  # 2 คือ เทรชโฮลด์ค่าที่ 1
4  # 3 คือ เทรชโฮลด์ค่าที่ 2
```

Canny

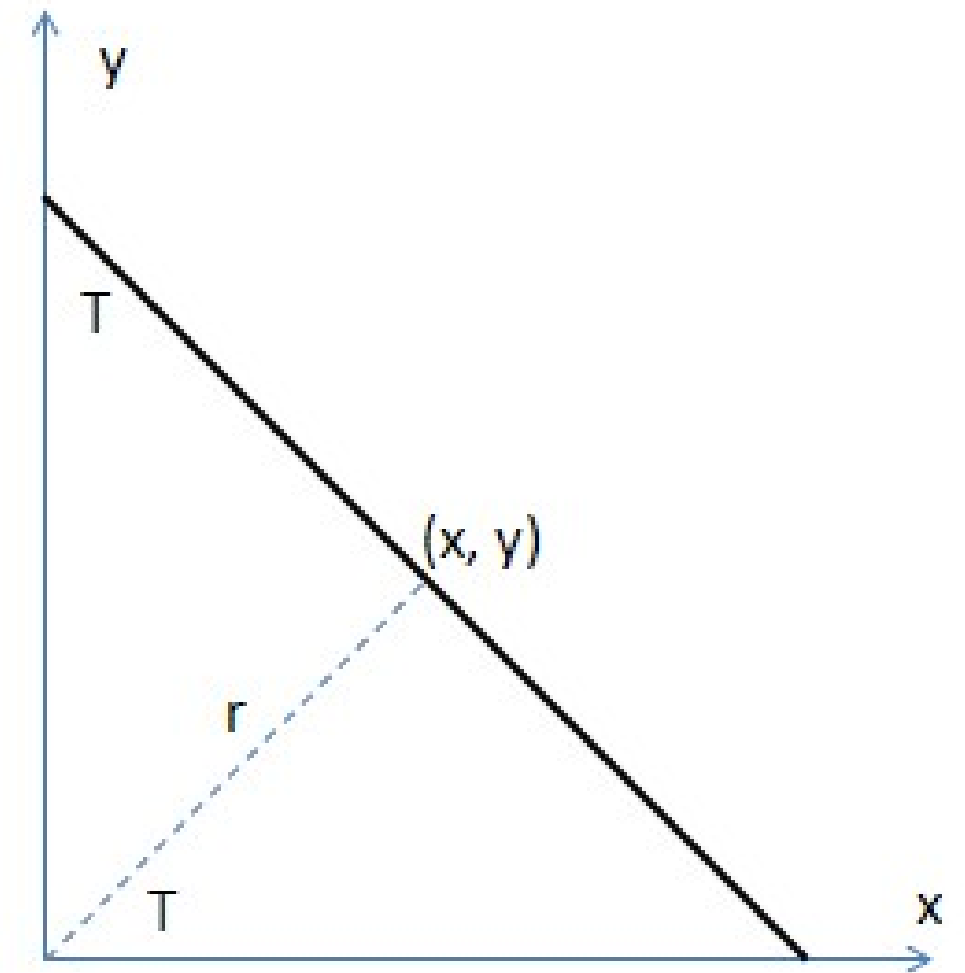


```
1  #Canny Method
2  import cv2
3
4  img = cv2.imread("image/currency.jpg",0)
5
6  # canny = cv2.Canny(1,2,3)
7  # 1 คือ รูปภาพ
8  # 2 คือ เทรชโฮลด์ค่าที่ 1
9  # 3 คือ เทรชโฮลด์ค่าที่ 2
10 canny = cv2.Canny(img,50,200)
11
12 cv2.imshow("Original",img)
13 cv2.imshow("Canny",canny)
14 cv2.waitKey(0)
15 cv2.destroyAllWindows()
```


Hough Transform

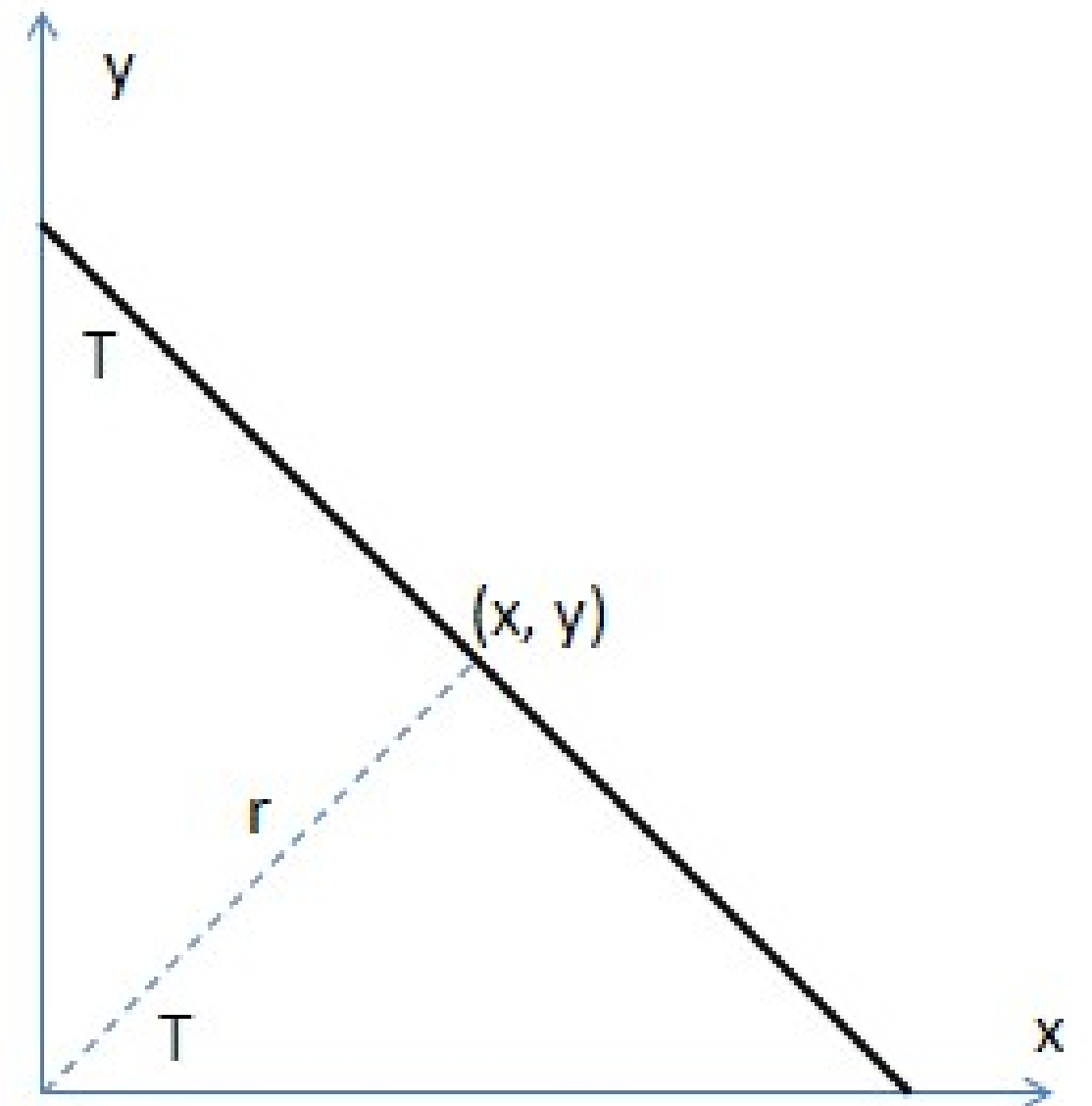
ถ้ามีเส้นตรงใดๆ (สีดำ) ตามรูป ที่มีระยะจากจุด $(0,0)$ ไปตั้งฉากกับเส้นตรงนั้นเท่ากับระยะ r โดยเส้นตั้งฉากนี้ทำมุมกับแกน x เป็นมุม T

เส้นตรงปกติจะมีสมการเป็น $y = mx + c$



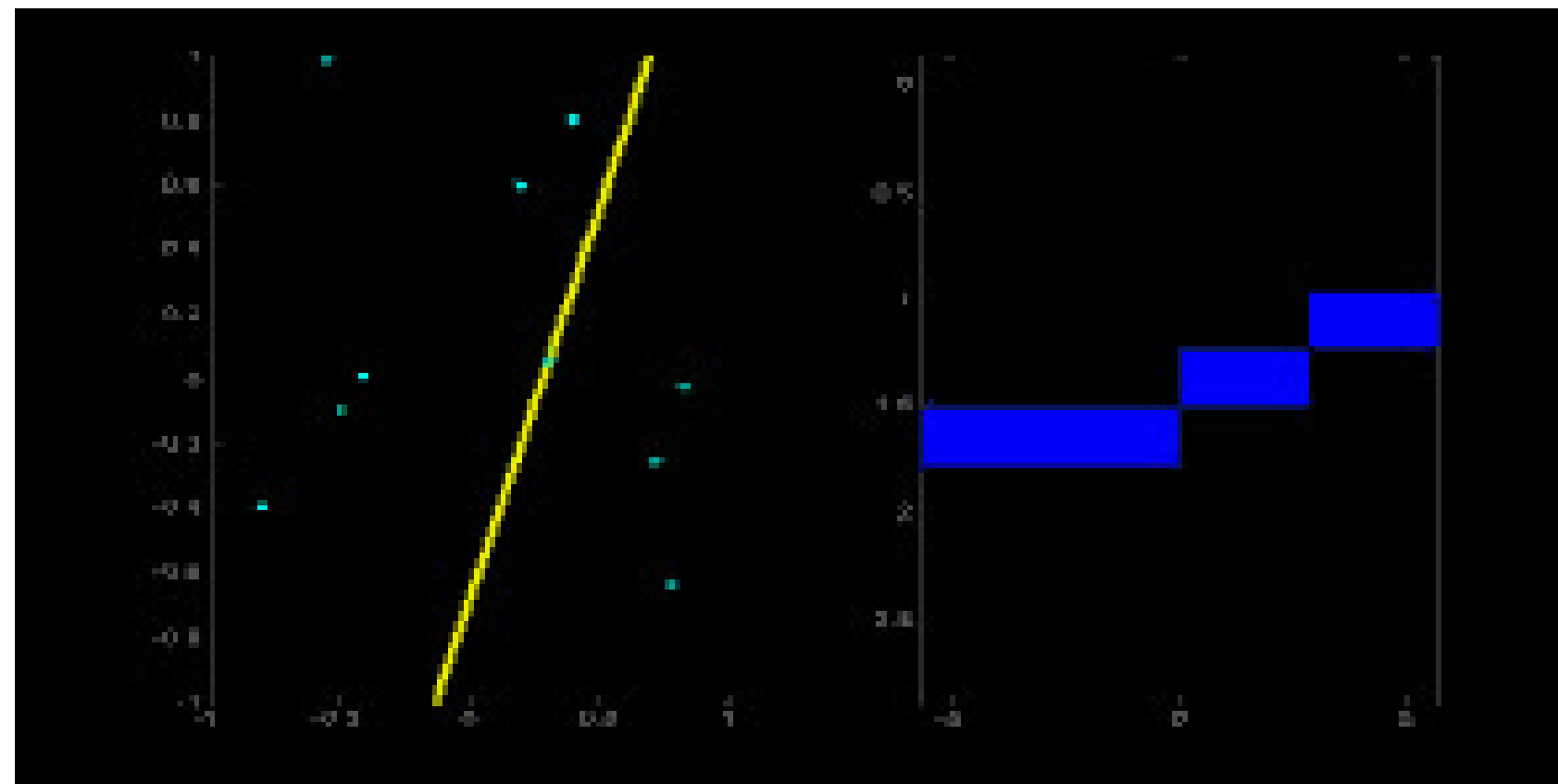
Hough Transform

$$\begin{aligned}y &= (-1/\tan(T)) x + (r/\sin(T)) \\y &= -\cos(T)/\sin(T) x + r/\sin(T) \\y \sin(T) &= -x \cos(T) + r \\y \sin(T) + x \cos(T) &= r\end{aligned}$$



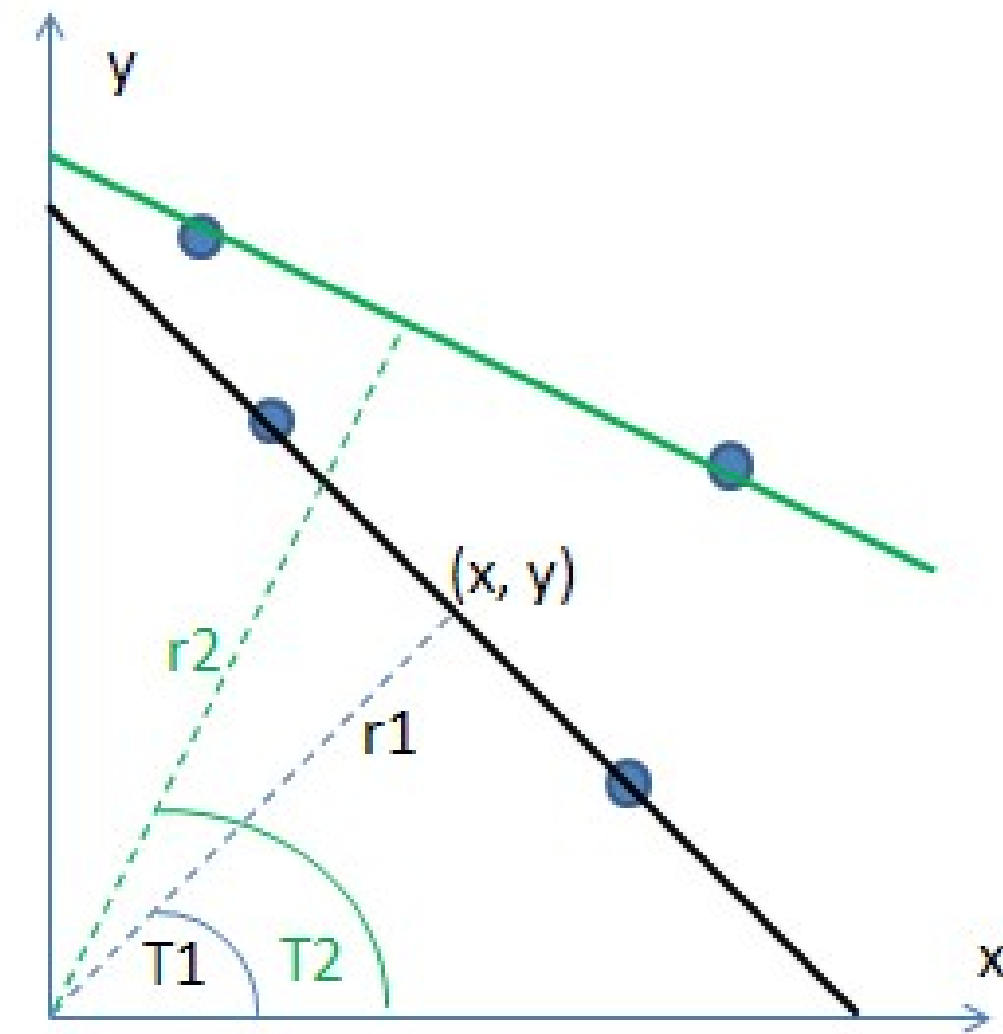
Hough Transform

มีจุดๆหนึ่ง เส้นตรงที่ลากผ่านจุดนี้จะมีได้ไม่จำกัด นั่นคือ จะมีค่า r และ T มากมาย
แต่ถ้ามีจุดสองจุดขึ้นไป เส้นตรงที่ลากผ่านจุดเหล่านี้จะมีได้เส้นเดียว นั่นคือ เส้นที่มีค่า r และ T เท่ากัน



Hough Transform

1. หาขอบของภาพด้วยวิธีใดๆ เช่น Canny ให้ได้ผลลัพธ์เป็นขาวดำ (ขาว = ขอบ)
2. เริ่มต้นด้วย $r=0$, $T=0$ (เส้นตรงที่ทับกับแกน y) ไล่ตรวจสอบว่ามีพิกเซลสีขาว (ขอบ) ที่อยู่บนแนวเส้นตรงนี้เท่าใด ถ้ามีเกินกว่าค่า threshold ที่กำหนด (เช่น 100 จุด) ก็ให้เก็บค่า (r, T) นี้ไว้
3. เปลี่ยนแปลงค่า r ไปเรื่อยๆ ตรวจสอบเช่นเดิม
4. เปลี่ยนแปลงค่า T ไปเรื่อยๆ ตรวจสอบเช่นเดิม
5. สุดท้ายจะได้ชุดของ (r, T) ซึ่งแต่ละคู่จะเป็นค่าของขอบเส้นตรงที่อยู่ในรูป
6. พล็อตเส้นตรงเหล่านั้น



Hough Transform



```
1  import cv2 as cv
2  import numpy as np
3
4  img = cv.imread('image/1.png')
5  gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
6  edges = cv.Canny(gray, 50, 150, apertureSize = 3)
7
8  lines = cv.HoughLines(edges, 1, np.pi/180, 200)
```


Hough Transform

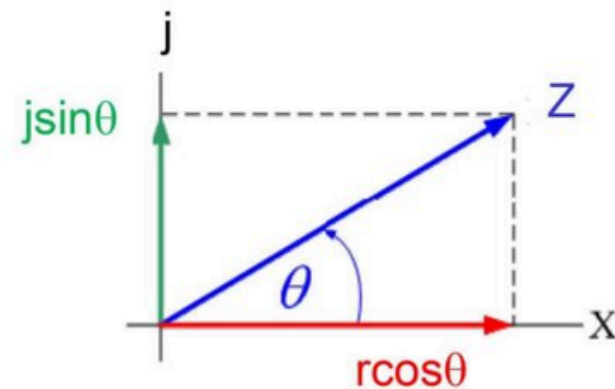
แบบตรีโกณมิติ (Trigonometric Form)

- เป็นรูปแบบของฟังก์ชันตรีโกณมิติโดยแท้ นั่นคือ มีค่า $\cos$ และ $\sin$ รวมอยู่ด้วย
- จะได้ว่า $X = r\cos\theta$
และ $Y = r\sin\theta$

จาก $z = x+jy$

จะได้ว่า $z = r\cos\theta + jr\sin\theta = r(\cos\theta + j\sin\theta)$

เขียนเป็นเวกเตอร์ได้ดังนี้



Hough Transform



```
1  for line in lines:
2      rho, theta = line[0]
3      a = np.cos(theta)
4      b = np.sin(theta)
5      x0 = a*rho
6      y0 = b*rho
7      x1 = int(x0 + 1000*(-b))
8      y1 = int(y0 + 1000*(a))
9      x2 = int(x0 - 1000*(-b))
10     y2 = int(y0 - 1000*(a))
11
12     cv.line(img, (x1,y1), (x2,y2), (0,0,255), 2)
```

Hough Transform



```
1  import cv2 as cv
2  import numpy as np
3
4  img = cv.imread('image/1.png')
5  gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
6  edges = cv.Canny(gray, 50, 150, apertureSize = 3)
7  lines = cv.HoughLinesP(edges, 1, np.pi/180, 100, minLineLength=100, maxLineGap=10)
8  for line in lines:
9      x1, y1, x2, y2 = line[0]
10     cv.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
11
12 cv.imshow("HoughLines", img)
13
14 cv.waitKey(0)
15 cv.destroyAllWindows()
```